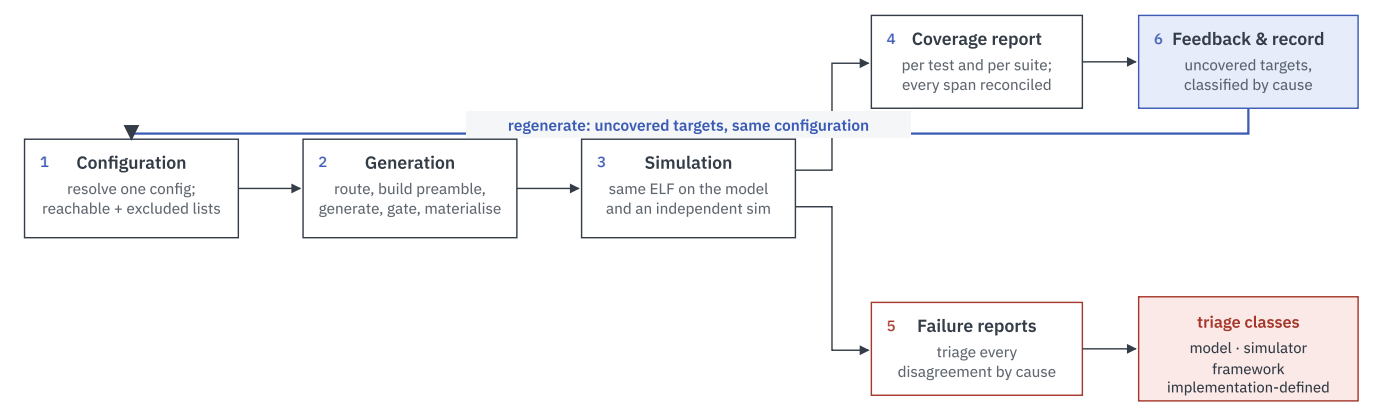


Sail Model to Coverage Report

Six stages in a closed loop. Coverage is not only a report — it is the feedback that drives generation of tests for what is still missing.



The whole pipeline runs separately for each of the seven configurations, and each reports its own coverage rather than a combined figure.

Coverage and failure reports are *parallel* outputs of simulation, not sequential: coverage comes from the instrumented model run, failure reports from comparing the two simulators' verdicts. A failing test writes no coverage data at all, so the two paths carry different tests.

Coverage: what “100%” means

Every target in the configuration's reachable denominator has at least one generated test, and every excluded target is recorded with a reason. It is a statement about the target list — not about Sail source branches, and not about differential agreement.

Reachable coverage	Does every reachable target have a test?
Model source coverage	Which Sail source branches were exercised?
Fault sensitivity	Can each test fail — proven by a passing and a deliberately broken run?
Differential agreement	Does a simulator not derived from the model agree?

Reported separately, never combined. One number cannot distinguish what was reached from whether the tests could detect a fault.

How the privileged architecture is reached

The model implements the entire privileged architecture through **nine instructions**. Coverage therefore comes from machine state the preamble builds, not from opcode selection — which is what makes 29 extensions and capabilities reachable at all.

SURFACE	MECHANISM	BECOMES A TARGET AS
M / S / U modes	Preamble sets <code>mstatus.MPP / SPP</code> ; returns via <code>MRET / SRET</code>	one per mode per behaviour
CSRs	Model-derived register set swept per instruction, at each legal privilege	one per (CSR, access, privilege)
Traps & exceptions	Preamble constructs the faulting condition; handler asserts both cause and origin	one per exception cause
Interrupts & delegation	Pending and enable bits set directly; <code>mideleg</code> toggled	one per cause × {taken, not taken, delegated}
Virtual memory	Preamble builds a page table and points <code>satp</code> at it	one per mode × PTE feature × fault class
PMP	Preamble programs entries with the correct order and lock bits	one per match type × access × privilege
PMA	Region attributes come from the model's configuration JSON	one per attribute, under a config declaring the access illegal

Committed scope

AREA	COMMITTED	DEFERRED – ARCHITECTURE READY	EXCLUDED
M-mode	CSR access, trap entry and return, privilege transitions, PMP, PMA, interrupt delivery and delegation	Nested traps	—
S-mode	All translation schemes — Sv32, Sv39, Sv48, Sv57, Svbare — with negative controls, plus PTE content features and the fault taxonomy	—	—
Privileged extensions	29 of 29	—	—
Unprivileged ISA	Only what privileged scenarios require	Systematic generation across the unprivileged ISA	—
Floating point	—	F/D instruction functionality. Configurations with F and D enabled are still generated and run	—
Vector	—	Vector instruction functionality, same reason and same route	—
Hypervisor	—	—	Not implemented in the model

Configurations

These seven configurations come from the Golden Model's own test suite. They are not a free XLEN × VLEN × ELEN cross-product; the model constrains which combinations are valid.

CONFIGURATION	XLEN	VLEN	ELEN	GENERATION ROUTE
rv32d_v64_e32	32	64	32	Symbolic
rv64d_v64_e64	64	64	64	Symbolic
rv32d_v128_e32	32	128	32	Symbolic
rv64d_v128_e64	64	128	64	Symbolic
rv32d_v256_e32	32	256	32	Oracle only
rv64d_v256_e64	64	256	64	Oracle only
rv64d_v512_e64	64	512	64	Oracle only

Each configuration has its own reachable target list, exclusion list and coverage result. Results are reported separately, not averaged. VLEN=256 and VLEN=512 use the concrete oracle because they exceed the symbolic engine's 129-bit value limit.

Deliverables

- 1 Repository with framework, user-facing and developer documentation
- 2 Example scripts generating a suite from a given configuration
- 3 Example scripts executing a suite, collecting results, summarising
- 4 CI integration into the Golden Model
- 5 Required Golden Model changes

Generation detail

Which backend serves which target, why two exist, and what each one structurally cannot do, are on the companion page — *ISLA vs Oracle*.